DESENVOLVIMENTO DE APLICATIVOS MOBILE



## MÓDULO 12

# Publicação na Google Play

Build de release, assinatura, App Bundle, Play Console, faixas de teste, verificação de desenvolvedor, LGPD e CI/CD

**Prof. Dr. Francisco**
IFMA – Campus São Luís – Monte Castelo
Departamento de Eletroeletrônica (DEE)

1 de novembro de 2026

# Objetivos do módulo

**Ao final deste módulo você será capaz de**

- Preparar um build de release otimizado e versionado.
- Assinar o app e entender a Play App Signing.
- Criar a conta e a ficha do app na Play Console, incluindo as declarações obrigatórias.
- Conduzir testes interno, fechado e aberto e cumprir a regra de 12 testadores.
- Lançar de forma gradual e acompanhar a saúde do app em produção.
- Atender à LGPD e automatizar a publicação com CI/CD.

# Roteiro do módulo

1. Preparando o release
2. Assinatura do app
3. Play Console
4. Faixas de teste
5. Lançamento e operação
6. LGPD e privacidade
7. Automatizando com CI/CD
8. Fechamento

# 1
# Preparando o release

# Build de release

app/build.gradle.kts

```
android {
  defaultConfig {
    versionCode = 7          // inteiro, sempre crescente
    versionName = "1.2.0"    // exibido ao usuário
  }
  buildTypes {
    release {
      isMinifyEnabled = true      // R8: encolhe e ofusca
      isShrinkResources = true    // remove recursos não
      usados
      proguardFiles(getDefaultProguardFile(
          "proguard-android-optimize.txt"),
        "proguard-rules.pro")
      signingConfig = signingConfigs.getByName("release")
    }
  }
}
```

**Versionamento**

`versionCode` identifica a versão para a loja: cada envio precisa de um número maior. `versionName` segue o *versionamento semântico* (maior.menor.correção).

**R8**

A ofuscação renomeia classes. Bibliotecas que usam reflexão podem precisar de regras `-keep`. **Teste o build release** antes de enviar.

**Dica**

Guarde o `mapping.txt` de cada versão: ele desofusca as pilhas de erro no Crashlytics e na Play Console.

# Checklist antes de publicar

**Produto**

- Ícone adaptativo (*Image Asset Studio*) e nome definitivos.
- `applicationId` definitivo (não muda depois).
- `targetSdk` atendendo à exigência vigente.
- Textos em `strings.xml`; tema claro e escuro revisados.
- Mensagens de erro, estados vazios e modo offline testados.

**Técnico**

- Build release testado em aparelhos reais (versões antigas e novas).
- Logs de depuração e URLs de teste removidos.
- Nenhum segredo no código ou no repositório.
- Crashlytics ativo e `mapping.txt` enviado.
- Apenas as permissões realmente usadas.

**Nível de API alvo**

Desde **31/08/2026**, apps novos e atualizações na Google Play precisam de `targetSdk` **36** (Android 16) ou superior. A exigência sobe todo ano, por volta de agosto.

# 2
# Assinatura do app

# Play App Signing




**Duas chaves**

A **chave do app** fica guardada pelo Google e assina o que chega aos usuários. A **chave de upload** fica com você e só prova ao Google que o envio é seu.

**Vantagem**

Se a chave de upload vazar ou for perdida, você pede a troca ao suporte da Play. Antes da Play App Signing, perder a chave significava nunca mais atualizar o app.

**Atenção**

O SHA-1 da chave do app (na Play Console) precisa ser cadastrado no Firebase para o login Google funcionar em produção (Módulo 09).

# Criando a chave e assinando

**Gerando a chave de upload**

```
keytool -genkeypair -v -keystore upload.jks \
  -alias upload -keyalg RSA -keysize 2048 \
  -validity 10000
```

**app/build.gradle.kts**

```
signingConfigs {
  create("release") {
    storeFile = file(System.getenv("KEYSTORE_PATH")
      ?: "upload.jks")
    storePassword = System.getenv("KEYSTORE_PASS")
    keyAlias = "upload"
    keyPassword = System.getenv("KEY_PASS")
  }
}
```

**Gerando o App Bundle**

```
./gradlew bundleRelease
# app/build/outputs/bundle/release/app-release.aab
```

**Nunca no Git**

O `.jks` e as senhas **não** vão para o repositório. Use variáveis de ambiente, `~/.gradle/gradle.properties` ou *secrets* da CI.

**Backup**

Guarde o `.jks` e as senhas num gerenciador de senhas e em uma cópia offline.

**Dica**

Pelo Android Studio: **Build** → **Generate Signed App Bundle / APK** faz o mesmo com um assistente.

# 3
# Play Console

# A conta de desenvolvedor

| | Pessoal | Organização |
|---|---|---|
| Para | pessoa física | empresas, instituições |
| Exige | verificação de identidade | D-U-N-S e verificação |
| Teste fechado obrigatório | sim (contas novas) | não |
| Nome exibido | seu nome | nome da organização |

**Custo**

Taxa de registro **única**, paga ao criar a conta em `https://play.google.com/console`.

**Para projetos institucionais**

Apps que representam o IFMA, um laboratório ou um projeto de extensão ficam melhor em uma **conta de organização** da instituição, que garante continuidade quando os alunos se formam.

**Atenção**

A conta exige verificação de identidade, telefone e e-mail de contato públicos. Mantenha esses dados atualizados: contas inativas ou com dados inválidos podem ser encerradas.

# Ficha da loja e declarações obrigatórias

**Ficha da loja**

- **Nome** do app (até 30 caracteres).
- **Descrição curta** (até 80) e **completa** (até 4000).
- **Ícone** 512×512 px.
- **Gráfico de recursos** 1024×500 px.
- **Capturas de tela** (celular e tablet).
- Categoria e e-mail de contato.

**Conteúdo do app (Política)**

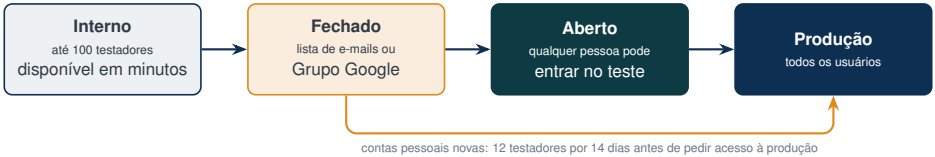
- URL da **política de privacidade**.
- Formulário de **segurança dos dados**.
- **Classificação de conteúdo** (questionário IARC).
- **Público-alvo** e se contém anúncios.
- Declaração de permissões sensíveis.
- Acesso para o revisor (se houver login).

**Atenção**

Se o app permite criar conta, ele deve oferecer **exclusão da conta** dentro do app e por um link na web.

# 4
# Faixas de teste

# Do teste interno à produção




**Regra dos 12 testadores**

Contas **pessoais** criadas após 13/11/2023 precisam manter um teste fechado com **pelo menos 12 testadores inscritos por 14 dias seguidos** antes de solicitar acesso à produção.

**Na prática**

- Recrute 15–20 testadores: se alguém sair, a contagem pode recomeçar.
- Uma turma do curso é um ótimo grupo de testadores!
- Publique atualizações durante o teste e registre o feedback.

# 5
# Lançamento e operação

# Lançamento gradual e saúde do app

**Lançamento gradual**

Libere a nova versão para 5 %, 20 %, 50 % e depois 100 % dos usuários, acompanhando as falhas a cada etapa. Se algo der errado, **interrompa** o lançamento.

**Android vitals**

A Play Console mostra a taxa de falhas e de ANRs percebidos pelos usuários. Acima dos limites de "mau comportamento", o app perde visibilidade na loja.

**Depois de publicado**

- Responda às avaliações: a resposta é pública.
- Use **In-App Updates** para pedir a atualização de versões críticas.
- Use **In-App Review** para pedir avaliação no momento certo.
- Leia as notas de política da Play: as regras mudam.
- Planeje a atualização anual do `targetSdk`.

# Distribuição fora da Play e verificação de desenvolvedor

**Novidade de 2026 no Brasil**

Desde **30/09/2026**, aparelhos Android certificados no Brasil (e também na Indonésia, em Singapura e na Tailândia) só instalam normalmente apps **registrados por desenvolvedores verificados**, inclusive quando distribuídos fora da Google Play (APK em site, lojas alternativas). A expansão global está prevista para 2027.

**Como se registrar**

Pela **Play Console** (apps na loja) ou pela **Android Developer Console** (apps só fora da loja), verificando a identidade e registrando o nome do pacote e a chave de assinatura.

**Estudantes e hobbistas**

Existem contas de **distribuição limitada**, pensadas para estudantes e projetos pessoais que instalam apps em poucos aparelhos.

**O que não muda para o desenvolvimento**

Instalar pelo Android Studio ou via **ADB** continua funcionando. Usuários avançados também têm um fluxo especial para instalar apps não verificados.

**Dica**

Para distribuir o LabMonitor só no laboratório via APK, registre o app antes.

# 6
# LGPD e privacidade

# LGPD no desenvolvimento de apps

**Lei Geral de Proteção de Dados (Lei 13.709/2018)**

Regula o tratamento de **dados pessoais** no Brasil: qualquer informação que identifique uma pessoa (nome, e-mail, matrícula, localização, identificadores do aparelho).

**Princípios-chave**

**Finalidade** (para quê), **necessidade** (o mínimo), **transparência** (explicar), **segurança** (proteger) e uma **base legal** (ex.: consentimento ou execução de contrato).

**Checklist para o app**

- Colete só o necessário (*privacy by design*).
- Política de privacidade clara, no app e na loja.
- Consentimento para usos opcionais (analytics).
- Acesso, correção e exclusão dos dados e da conta.
- HTTPS, criptografia e regras no backend.

**Atenção**

Não é assessoria jurídica: em projetos institucionais, consulte o encarregado de dados (DPO).

# 7
# Automatizando com CI/CD

# Publicação automática

**.github/workflows/release.yml (trecho)**

```yaml
on:
  push:
    tags: ["v*"]          # ex.: git tag v1.2.0
jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 17 }
      - name: Restaurar keystore
        run: echo "$KEYSTORE_B64" | base64 -d > upload.jks
        env: { KEYSTORE_B64: "${{ secrets.KEYSTORE_B64 }}" }
      - name: Gerar AAB
        run: ./gradlew bundleRelease
        env:
          KEYSTORE_PASS: ${{ secrets.KEYSTORE_PASS }}
          KEY_PASS: ${{ secrets.KEY_PASS }}
      # enviar à faixa interna (Gradle Play Publisher
      # ou Fastlane supply) com uma conta de serviço
```

**Pipeline típico**

*push* de uma tag → testes → build assinado → envio à faixa interna → promoção manual para produção.

**Ferramentas**

**Gradle Play Publisher** (plugin Gradle) ou **Fastlane** (`supply`) usam a API da Play Console com uma *conta de serviço* do Google Cloud.

**Atenção**

Segredos ficam nos *secrets* do repositório, nunca no YAML.

# 8
# Fechamento

# Resumo

**Resumo**

- Release = R8, `versionCode` crescente, `targetSdk` atualizado e testes no build final.
- Play App Signing: você guarda a chave de upload; o Google guarda a chave do app.
- A loja exige ficha completa, política de privacidade, segurança dos dados e classificação.
- Faixas interna, fechada e aberta antes da produção; contas pessoais novas precisam de 12 testadores por 14 dias.
- No Brasil, desde 30/09/2026, apps precisam ser de desenvolvedores verificados mesmo fora da Play.
- LGPD: mínimo de dados, transparência, consentimento e exclusão. CI/CD automatiza tudo.

# Exercícios

**Exercício 1 — Release**

Gere o AAB assinado do LabMonitor, instale o build release com `bundletool` e verifique se tudo funciona com o R8 ativo.

**Exercício 2 — Política**

Escreva a política de privacidade do LabMonitor e preencha um rascunho do formulário de segurança dos dados com base no que o app realmente coleta.

**Exercício 3 — Ficha**

Produza ícone, gráfico de recursos, 4 capturas de tela e as descrições curta e completa do app.

**LabMonitor — Etapa 12**

Publique o app na faixa de **teste fechado** com a turma como testadores, colete o feedback por 14 dias e lance a versão 1.0.

# Para aprofundar

- Preparar o app para lançamento: `https://developer.android.com/studio/publish/preparing`
- Assinatura de apps: `https://developer.android.com/studio/publish/app-signing`
- Requisitos de nível de API:
  `https://developer.android.com/google/play/requirements/target-sdk`
- Testes para contas pessoais novas:
  `https://support.google.com/googleplay/android-developer/answer/14151465`
- Verificação de desenvolvedor: `https://developer.android.com/developer-verification`
- LGPD (texto da lei):
  `https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm`

# Fim do Módulo 12

**Próximo:** Módulo 13 — Desenvolvimento multiplataforma: Flutter, React Native e Kotlin Multiplatform.